

UFMG



TRABALHO PRÁTICO 2

Fundamentos de Inteligência Artificial
Outubro de 2026

Professores: Cristiano Leite de Castro e João Pedro Campos

Busca Local e Algoritmo Genético no Sequenciamento de Manutenção de Ativos

Integrantes do Grupo: Marcus Vinicius Ferreira Vilela
Pedro
Thales
Vitor Lima

Universidade Federal de Minas Gerais (UFMG)

1 Introdução e modelagem

Este trabalho compara três abordagens para o sequenciamento da manutenção preventiva de n ativos por uma única equipe: *greedy descent* com *random restart* (GD), *simulated annealing* (SA) e um *genetic algorithm* (GA) [1, 3, 4]. Uma solução é uma permutação $\pi = (\pi_1, \ldots, \pi_n)$ (a ordem de visita), e o custo a minimizar é o atraso ponderado total

$$h(\pi) = \sum_{j=1}^{n} w_j \max(0,\, C_j - d_j), \qquad C_{\pi_k} = C_{\pi_{k-1}} + t_{\pi_{k-1}\pi_k} + p_{\pi_k}, \tag{1}$$

com deslocamentos t_{ij} iguais à distância euclidiana arredondada para o inteiro mais próximo. As três instâncias têm $n = 12$, 50 e 150 ativos.

Arquitetura. O código separa algoritmo e problema (`src/base.py`). Os algoritmos só chamam `solucao_aleatoria`, `h`, `sucessores`, `sucessor_aleatorio`, `aplicar`, `cruzamento`, `mutacao` e `distancia`. Toda avaliação de h passa por um único método (`_contar`), que incrementa o contador, lança uma exceção quando o orçamento B é atingido (a execução para *exatamente* em B) e registra a curva de convergência. Os três algoritmos não contêm nenhuma referência a ativos, prazos ou tempos.

T1: validação de h. A saída do programa (`python3 src/experimentos.py t1`) reproduz os valores de verificação:

```
pequena: h(identidade) = 4366
pequena: guloso (menor acréscimo de atraso) = 3168
pequena: ótimo exato (PD com fronteira de Pareto) = 1875,
         pi* = (5, 4, 1, 6, 7, 2, 8, 12, 3, 10, 9, 11)
```

O exemplo da Figura 1 do enunciado ($h = 39$ e $h^\star = 22$) também é verificado em teste automático (`tests/test_manutencao.py`). Dois detalhes merecem registro:

- o guloso só reproduz 3168 quando os empates em "menor acréscimo de atraso" (frequentes no início, quando o acréscimo é zero) são desfeitos pelo ativo *mais próximo*; desempatando pelo menor índice, ele dá 6935;
- o ótimo 1875 foi confirmado de forma independente por programação dinâmica sobre (conjunto visitado, último ativo), mantendo a fronteira de Pareto de (tempo, custo). Essa busca exata mostrou que o ótimo global é único.

2 Vizinhanças e avaliação incremental (T2)

Os três operadores (*swap*, *insertion* e *2-opt*) induzem vizinhanças de tamanho $n(n-1)/2$, $(n-1)^2$ e $n(n-1)/2$, respectivamente. Na *insertion*, mover o ativo da posição i para $i+1$ produz a mesma sequência que mover o de $i+1$ para i; descontadas essas $n-1$ duplicatas, sobram $(n-1)^2$ vizinhos distintos, o que foi conferido em teste. Para $n = 150$ isso dá $11\,175$, $22\,201$ e $11\,175$ sucessores.

Avaliação incremental. Um movimento que altera apenas as posições $a..b$ mantém o prefixo $\pi_1..\pi_{a-1}$, cujo custo e tempo de conclusão são lidos do cronograma já calculado para π. Recalcula-se só o trecho $a..b$. Da posição $b+1$ em diante os ativos são os mesmos, e todos sofrem o mesmo deslocamento de tempo δ. Assim, o custo do sufixo é $\sum_{k>b} w_k \max(0, \delta - f_k)$, com folgas $f_k = d_k - C_k$ pré-calculadas; se $\delta = 0$, o sufixo custa o mesmo que antes. Como h é não decrescente ao longo do prefixo, a soma é interrompida quando ultrapassa um limite (o h da solução corrente). Um valor interrompido só é usado para rejeitar o movimento, nunca é reportado.

Um teste automático compara a avaliação incremental com a completa em 5400 movimentos das três vizinhanças, nas três instâncias, com e sem limite. Dentro de uma descida *first improvement*, a avaliação incremental reduziu o tempo por sucessor de $12\,\mu s$ para $8\,\mu s$ em $n = 150$ (ganho de cerca de $1{,}5\times$). Em $n = 12$ não houve ganho: o custo fixo do Python domina e o sufixo é curto demais para compensar (Tabela 1). A contagem de avaliações não muda, e verificamos que a trajetória da busca é idêntica nos dois modos.

Tabela 1: T2: GD com *random restart* em cada vizinhança (10 execuções, mesmo orçamento da Seção 3). A coluna p é o teste de Mann–Whitney contra a melhor vizinhança na variante *first*. As colunas de tempo medem o custo por sucessor dentro de uma descida *first improvement*.

		mediana do melhor h (10 exec.)			μs por sucessor		
Instância	Vizinhança	Best	First	p (First)	completa	incremental	ganho
Pequena ($n = 12$)	Swap	1 875	**1 875**	–	1,7	1,6	1,02×
	Insertion	1 875	1 875	1,000	1,6	1,5	1,06×
	2-opt	1 875	1 875	1,000	1,7	1,6	1,03×
Intermediária ($n = 50$)	Swap	30 896	25 690	0,0002	4,6	3,0	1,51×
	Insertion	20 326	**17 408**	–	4,4	3,1	1,40×
	2-opt	20 514	18 502	0,121	4,4	3,0	1,45×
Grande ($n = 150$)	Swap	468 855	150 198	0,0002	12,7	8,3	1,54×
	Insertion	821 542	**81 175**	–	11,9	8,1	1,48×
	2-opt	473 487	95 346	0,0003	11,9	7,6	1,56×

Na instância pequena as três vizinhanças empatam (todas chegam a 1875). Nas maiores, a ***insertion*** produz os melhores ótimos locais com *first improvement*:

- $n = 50$: mediana $17\,408$, contra $18\,502$ do *2-opt* e $25\,690$ do *swap*;
- $n = 150$: mediana $81\,175$, contra $95\,346$ e $150\,198$, com $p < 0{,}001$ nas duas comparações.

A explicação está na estrutura do problema. O custo depende de *quando* cada ativo é atendido, e a *insertion* adianta ou atrasa um único ativo deslocando os demais de apenas uma posição. O *swap* troca dois ativos de lugar e altera ao mesmo tempo dois deslocamentos de cada lado. O *2-opt* inverte um trecho inteiro, o que destrói a ordem por prazo que as boas soluções têm.

Por sucessor, as três vizinhanças custam praticamente o mesmo ($7{,}6\,\mu s$ a $8{,}3\,\mu s$ em $n = 150$).

Por *varredura completa*, porém, a *insertion* custa o dobro, e isso a torna a pior opção para *best improvement* em $n = 150$: a mediana chega a $821\,542$, porque cada passo consome $22\,201$ avaliações. **Adotamos a *insertion* nos demais experimentos.**

3 Algoritmos e parâmetros

(A) GD com *random restart*. Implementamos as três variantes:

- *best improvement*: varre toda a vizinhança e move para o melhor sucessor;
- *first improvement*: varre a vizinhança em ordem aleatória e aceita o primeiro sucessor que melhora;
- *sucessor aleatório*: sorteia um sucessor e o aceita se melhorar. Como essa variante não consegue provar que está num ótimo local, declara-se ótimo local após $|N(\pi)|$ sorteios seguidos sem melhora.

Ao atingir o ótimo local, recomeça-se de uma permutação aleatória.

(B) SA. Aceita π' se $h(\pi') \le h(\pi)$; caso contrário, aceita com probabilidade $e^{-\Delta/T}$ [3]. O resfriamento é geométrico, com $T \leftarrow \alpha T$ a cada avaliação, e os três parâmetros são definidos assim:

- T_0 é calibrado *na própria instância*: sorteiam-se 200 sucessores da solução inicial (essas avaliações contam no orçamento) e escolhe-se T_0 tal que a piora média $\bar{\Delta}^+$ seja aceita com probabilidade χ_0, isto é, $T_0 = -\bar{\Delta}^+ / \ln \chi_0$. Isso resolve o problema de escala apontado no enunciado: o T_0 mediano foi 941 na pequena, $14\,822$ na intermediária e 5852 na grande;
- T_f é absoluto. Como h é inteiro, a menor piora possível é 1, e com $T_f \le 1$ uma piora de 5 é aceita com probabilidade menor que $0{,}7\%$, ou seja, o final é praticamente *greedy descent*;
- $\alpha = (T_f/T_0)^{1/B'}$, em que B' é o orçamento restante, de modo que o resfriamento termine exatamente no fim do orçamento.

$\chi_0 \in \{0{,}2; 0{,}5; 0{,}8\}$ e $T_f \in \{0{,}2; 1; 5\}$ foram escolhidos numa calibração com 5 execuções por par e sementes distintas das do experimento principal (Anexo A). Resultado: $\chi_0 = 0{,}8$ e $T_f = 1$ na intermediária, $\chi_0 = 0{,}2$ e $T_f = 0{,}2$ na grande, e $\chi_0 = 0{,}5$ e $T_f = 1$ na pequena, onde todas as configurações empatam.

(C) GA. O algoritmo é geracional, com elitismo de 2 indivíduos, $N = 50$, $P_c = 0{,}9$ e $P_m = 0{,}5$, e a população inicial é aleatória. Os operadores são:

- **seleção:** torneio de tamanho 3. Como o torneio só compara indivíduos, basta declarar vencedor o de *menor* h; não é preciso converter custo em aptidão, ao contrário da roleta, que exigiria algo como $f = 1/(1+h)$ ou $f = h_{\max} - h$;
- **cruzamento:** *Order Crossover* (OX) [1, 2], detalhado abaixo;

- **mutação:** por *insertion*, que na calibração (Anexo A) foi muito superior à *swap mutation* em todas as instâncias. Por exemplo, a mediana em $n = 50$ foi $17\,704$ contra $26\,022$.

Um filho que não sofreu cruzamento nem mutação herda o h do pai sem gastar avaliação.

Exemplo do OX (cortes nas posições 4 e 7): pais $p_1 = (1\,2\,3\,\mathbf{4}\,\mathbf{5}\,\mathbf{6}\,\mathbf{7}\,8\,9)$ e $p_2 = (9\,3\,7\,8\,2\,6\,5\,1\,4)$.

1. O filho copia o trecho de p_1 nas mesmas posições: $(___\,4\,5\,6\,7\,__)$.
2. Lê-se p_2 a partir da posição seguinte ao 2.º corte, circularmente: $1\,4\,9\,3\,7\,8\,2\,6\,5$.
3. Removem-se os genes já presentes (4, 5, 6 e 7), o que deixa $1\,9\,3\,8\,2$.
4. Esses genes preenchem as posições livres a partir da 8, também circularmente: posição 8 $\leftarrow 1$, 9 $\leftarrow 9$, 1 $\leftarrow 3$, 2 $\leftarrow 8$, 3 $\leftarrow 2$.

O filho é $(3\,8\,2\,4\,5\,6\,7\,1\,9)$, uma permutação válida que preserva o bloco de p_1 e a *ordem relativa* de p_2. Esse exemplo é um teste automático do código.

4 Resultados (T3)

Seguimos o protocolo da Seção 3: orçamento de $50\,000$, $150\,000$ e $300\,000$ avaliações, com 30, 30 e 10 execuções independentes por combinação. Cada execução tem semente própria, e as mesmas sementes são usadas por todos os algoritmos. Reportamos as três variantes do GD; nas figuras, "GD" se refere ao *first improvement*, que é a melhor variante.

Tabela 2: Melhor h por combinação. "Gap" é $(h - h^\star)/h^\star$ com $h^\star = 1875$. "Passos" são os movimentos de melhoria aceitos (mediana). Tempo médio de parede por execução; os 8 processos rodaram em paralelo.

Instância	Algoritmo	média ± desvio	mínimo	mediana	gap médio	tempo (s)	passos
Pequena ($n = 12$)	GD best	1 875 ± 0	1 875	1 875	0,0%	0,1	363
	GD first	1 875 ± 0	1 875	1 875	0,0%	0,1	2 396
	GD aleatório	1 875 ± 0	1 875	1 875	0,0%	0,1	2 519
	SA	1 875 ± 0	1 875	1 875	0,0%	0,2	–
	GA	2 002 ± 154	1 875	1 937	6,8%	1,0	–
Intermediária ($n = 50$)	GD best	21 111 ± 2 262	17 451	20 690	–	0,7	61
	GD first	17 102 ± 584	16 234	16 920	–	0,7	1 687
	GD aleatório	17 392 ± 757	16 242	17 466	–	0,8	1 775
	SA	16 831 ± 507	16 235	16 681	–	1,2	–
	GA	19 548 ± 1 967	16 498	19 636	–	5,6	–
Grande ($n = 150$)	GD best	817 733 ± 24 398	788 457	809 608	–	3,4	13
	GD first	82 448 ± 7 270	73 413	82 655	–	3,2	1 226
	GD aleatório	81 547 ± 6 230	71 109	82 441	–	3,5	1 272
	SA	71 812 ± 5 847	64 519	71 064	–	4,3	–
	GA	89 739 ± 8 316	76 266	91 501	–	22,4	–

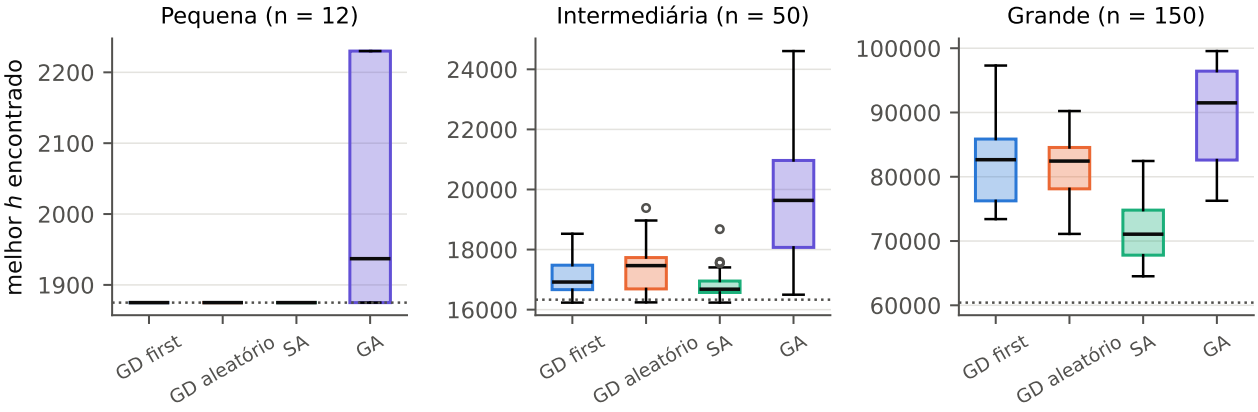


Figura 1: Distribuição do melhor h por algoritmo. A linha pontilhada é o valor de referência (ótimo em $n = 12$; melhor conhecido nas demais). O GD *best* fica fora da escala em $n = 150$ ($817\,733$ em média).

Tabela 3: Testes de Mann–Whitney bilaterais sobre o melhor h. Entre parênteses, o sinal da comparação entre as medianas (primeiro algoritmo versus segundo). "Idênticos" indica que todas as execuções deram 1875.

Comparação	Pequena ($n = 12$)	Intermediária ($n = 50$)	Grande ($n = 150$)
SA vs GD first	idênticos	0,048 (<)	0,0028 (<)
SA vs GD aleatório	idênticos	0,0018 (<)	0,0058 (<)
GD first vs GD aleatório	idênticos	0,160 (<)	0,910 (>)
GD first vs GD best	idênticos	$< 10^{-4}$ (<)	0,0002 (<)
SA vs GA	$< 10^{-4}$ (<)	$< 10^{-4}$ (<)	0,0006 (<)
GD first vs GA	$< 10^{-4}$ (<)	$< 10^{-4}$ (<)	0,054 (<)

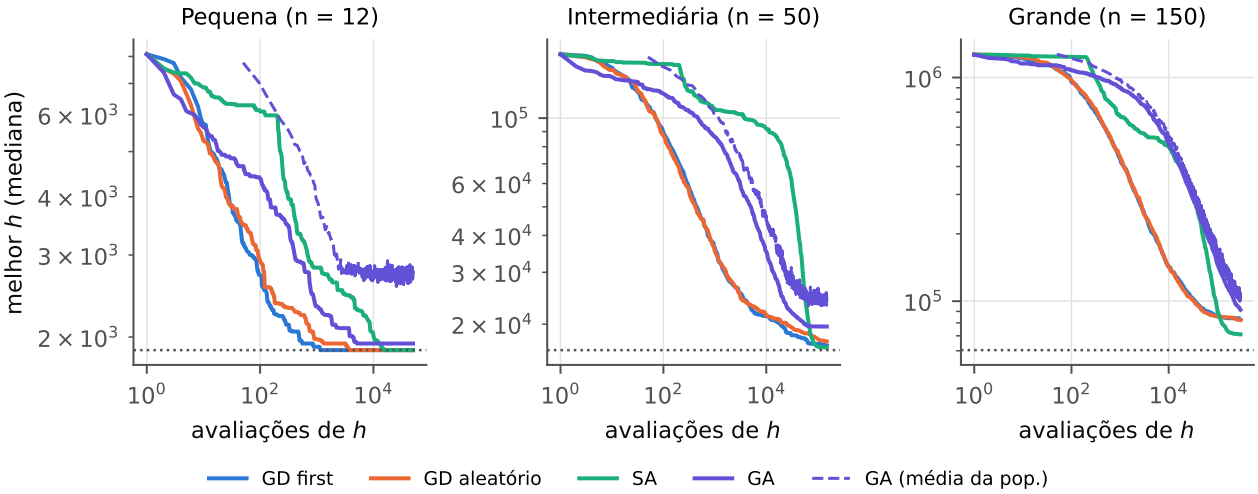


Figura 2: Curvas de convergência: mediana, entre execuções, do melhor h em função das avaliações (eixos logarítmicos). A curva tracejada é a mediana da *média da população* do GA.

Tempo $\times$ avaliações. Com o mesmo orçamento de avaliações, o GA gasta 22 s por execução

em $n = 150$, contra 3–4 s da busca local e do SA (Tabela 2). São cerca de $75\,\mu s$ por avaliação no GA contra $11\,\mu s$ no GD. A diferença tem três causas:

- cada filho do GA exige uma avaliação *completa* $O(n)$, sem a incremental;
- o cruzamento OX também custa $O(n)$;
- a diversidade da população é medida a cada geração.

Contar avaliações iguala o acesso à função objetivo, mas não iguala o custo computacional. Se o critério fosse tempo de parede, a desvantagem do GA seria ainda maior.

5 Discussão

Ótimos locais. Na instância pequena, as três variantes de GD encontraram $h^\star = 1875$ nas **30 de 30** execuções; o SA também. O GA só o encontrou em 11 de 30 execuções (gap médio de 6,8%).

- **GD first:** ao longo dos 4088 *restarts* (somando as 30 execuções), apareceram apenas **19 ótimos locais distintos** (17 valores distintos de h), e só 15% dos *restarts* terminaram no ótimo global (Figura 3).
- **GD best:** 12,7% de 1453 *restarts* terminaram no ótimo global, nos mesmos 19 ótimos locais.
- **GD aleatório:** declarou 447 "ótimos" distintos. A maioria não é ótimo local de verdade: a probabilidade de um sucessor específico nunca ser sorteado em $|N| = 121$ sorteios com reposição é $(1 - 1/121)^{121} \approx 37\%$, então a descida pára antes do fundo da bacia.

É a situação descrita em aula: o GD fica preso ao ótimo local da bacia em que começa, e o *random restart* só funciona porque a bacia do ótimo global tem cerca de 15% de probabilidade e há orçamento para mais de 100 reinícios por execução. O ótimo global encontrado é sempre a mesma permutação, o que confirma a unicidade obtida pela busca exata.

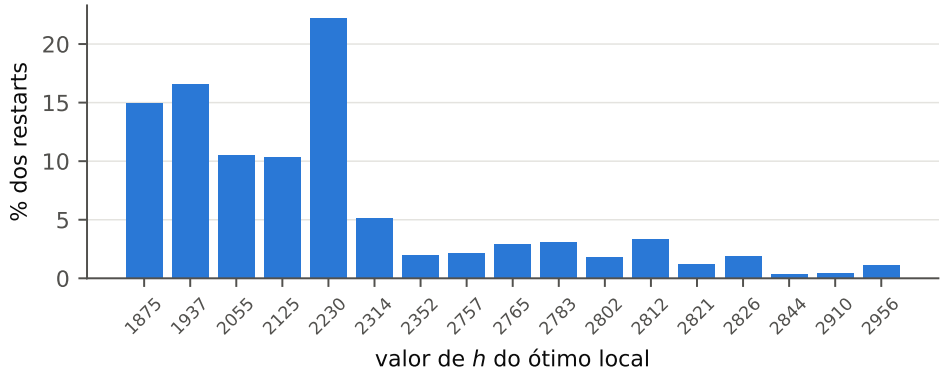


Figura 3: Valores de h dos ótimos locais atingidos pelo GD *first* na instância pequena (4088 *restarts* somados nas 30 execuções).

Exploração $\times$ intensificação.

- **GD.** A exploração vem só do *random restart*, e a intensificação, da descida. Em $n = 12$ esse equilíbrio é adequado, com 136 reinícios por execução. Em $n = 150$ ele colapsa: o GD *first* gasta todo o orçamento em praticamente uma única descida (1 reinício, 1226 passos), e o GD *best* não termina nem a primeira (13 passos). Nesse caso, a exploração não existe.

- **SA.** A exploração vem da aceitação de pioras quando T é alta, e a intensificação, do regime frio no final. A Figura 4 mostra a transição: a aceitação de pioras começa em 49%, 78% e 28% (pequena, intermediária e grande) e cai a 0% antes da metade do orçamento, de modo que a segunda metade é *greedy descent*. Na instância grande, a calibração escolheu $\chi_0 = 0{,}2$, ou seja, começar *longe* do *random walk*. Com $300\,000$ avaliações para $n = 150$, gastar o início em passeio aleatório deixa pouco orçamento para intensificar; o $\chi_0 = 0{,}8$ teve mediana $69\,715$, contra $67\,586$ (Anexo A). Ainda assim, a diferença entre as configurações é pequena diante da dispersão com 5 execuções.

- **GA.** A exploração vem da mutação e da população, e a intensificação, da seleção e do elitismo. A Figura 5 mostra que **não houve convergência prematura**: ao fim do orçamento ainda há 30–35 cromossomos distintos em 50, com distância de Hamming posicional média de cerca de 0,48. Na Figura 2, a média da população fica entre 11% e 35% acima do melhor indivíduo. O desequilíbrio é o oposto: a exploração é excessiva e a intensificação, fraca. O melhor h estaciona (na pequena, em 1937, o 2.º ótimo local mais frequente) enquanto a população continua diversa, e os filhos gerados com $P_m = 0{,}5$ quase sempre pioram. Taxas de mutação menores chegaram a ser testadas na calibração e não melhoraram o resultado.

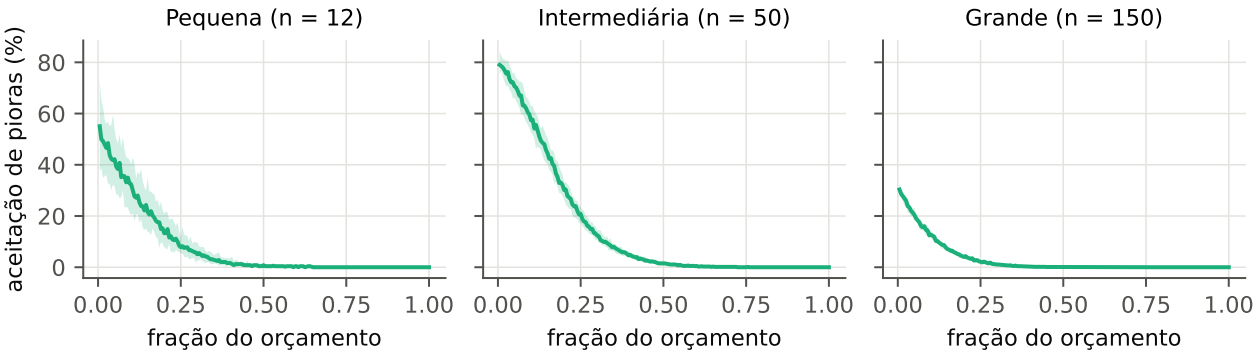


Figura 4: Taxa de aceitação de movimentos piores no SA, em janelas de 0,5% do orçamento: mediana entre execuções, com a faixa entre os percentis 10 e 90.

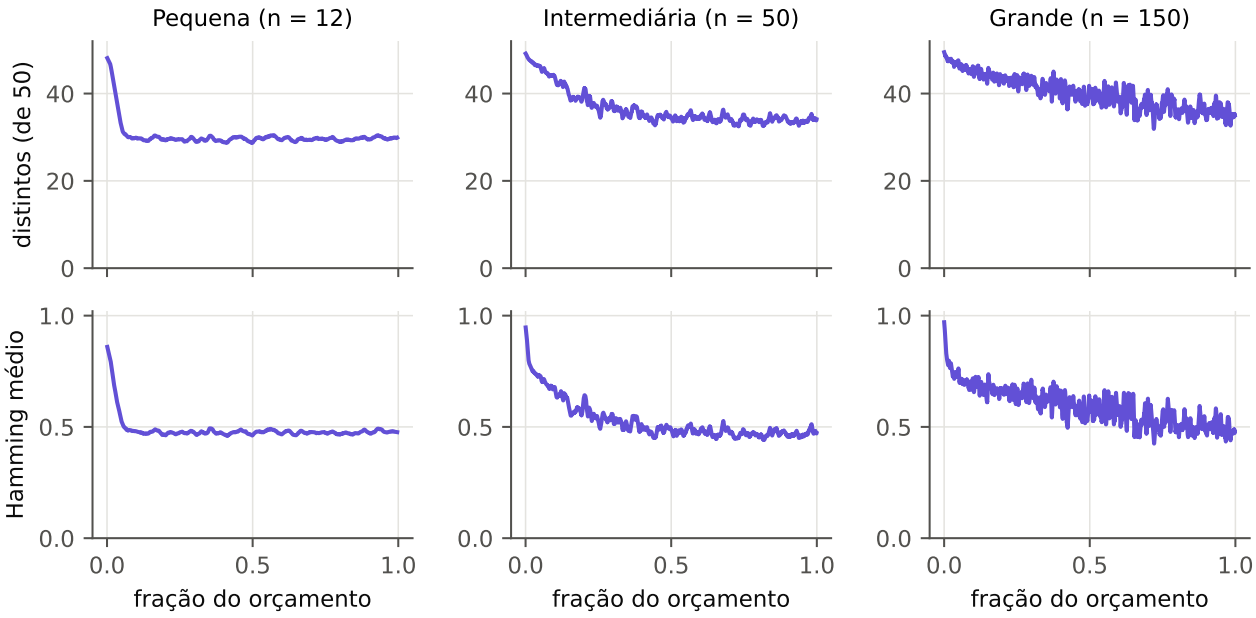


Figura 5: Diversidade da população do GA: número de cromossomos distintos e distância de Hamming posicional média (normalizada) entre 200 pares sorteados. Mediana entre execuções, com média móvel de 25 gerações.

Efeito da escala. O *ranking* muda com o tamanho da instância.

- **Pequena:** GD (qualquer variante) e SA empatam no ótimo, e o GA fica atrás ($p < 10^{-4}$).
- **Intermediária:** o SA supera o GD *first* ($p = 0{,}048$) e o GD aleatório ($p = 0{,}002$); o GD *best* cai para o mesmo patamar do GA.
- **Grande:** o SA vence com folga (mediana $71\,064$ contra $82\,655$, $p = 0{,}003$), o GD *first* e o aleatório empatam ($p = 0{,}91$), o GA *deixa de ser significativamente pior* que o GD ($p = 0{,}054$) e o GD *best* é, de longe, o pior.

Nossa hipótese é que a escala atua por dois mecanismos:

1. o custo de uma varredura cresce como n^2 e o orçamento só dobra de $n = 50$ para $n = 150$. A busca local que varre a vizinhança passa a caber em poucos passos (o *best* dá 13), e a que não varre perde o *restart*, que era o seu único mecanismo de exploração;
2. com o espaço de busca muito maior ($150!$), o mecanismo de fuga *dentro* de uma única trajetória, que é o do SA, passa a valer mais que reinícios independentes.

O empate estatístico entre GA e GD em $n = 150$ é efeito da piora do GD com a escala, e não de uma melhora do GA.

Quando o GA compensa? Em nenhuma instância o GA superou as duas abordagens de busca local; ele perde para o SA em todas ($p \leq 0{,}0006$). Formulamos a hipótese de que a recombinação *não* transmite bons trechos de sequência nesta codificação. O custo de um ativo depende do seu

instante de atendimento, isto é, de tudo o que vem antes dele. O OX preserva um bloco de p_1 na posição original e a ordem relativa de p_2, mas os genes de p_2 mudam de posição absoluta e, portanto, de instante: um trecho bom em p_2 chega ao filho com outro prefixo e outro custo.

Para testar a hipótese, repetimos o GA com o protocolo completo em três versões: com OX, com PMX (que mantém mais genes de p_2 na mesma posição absoluta) e **sem cruzamento** ($P_c = 0$, só mutação e seleção).

Tabela 4: Experimento adicional: efeito do operador de cruzamento no GA (mesmo protocolo da Seção 3, mesmas sementes).

Instância	GA	média $\pm$ desvio	mediana	p vs sem cruzamento
Pequena ($n = 12$)	OX	2 002 $\pm$ 154	1 937	0,707
	PMX	1 937 $\pm$ 120	1 875	0,015
	sem cruzamento ($P_c = 0$)	2 017 $\pm$ 162	1 937	–
Intermediária ($n = 50$)	OX	19 548 $\pm$ 1 967	19 636	0,469
	PMX	19 639 $\pm$ 2 229	19 212	0,501
	sem cruzamento ($P_c = 0$)	20 001 $\pm$ 2 058	19 496	–
Grande ($n = 150$)	OX	89 739 $\pm$ 8 316	91 501	0,121
	PMX	91 249 $\pm$ 10 311	89 978	0,121
	sem cruzamento ($P_c = 0$)	84 131 $\pm$ 5 818	83 983	–

A Tabela 4 sustenta a hipótese:

- nas instâncias intermediária e grande, nem o OX nem o PMX melhoram significativamente o GA sem cruzamento ($p \geq 0{,}12$). Em $n = 150$ a versão *sem* cruzamento tem até a melhor mediana ($83\,984$, contra $91\,501$ com OX). Ali o cruzamento consome orçamento sem contribuir, e o GA funciona como uma busca local populacional cara;
- o único ganho significativo é o do PMX na instância pequena ($p = 0{,}015$), coerente com o argumento: entre os dois operadores, é o que mais preserva posições absolutas.

Para o GA compensar, a recombinação precisaria respeitar o que determina o custo: operadores baseados em posição ou instante (PMX, *position-based crossover*), ou uma hibridização com a busca local (algoritmo memético). Ambos ficam como trabalhos futuros.

6 Conclusões

Com orçamento fixo de avaliações:

- o **SA** foi o melhor algoritmo nas instâncias intermediária e grande, com diferença estatisticamente significativa, e empatou no ótimo na pequena;
- o **GD** com *random restart* resolve a instância pequena de forma trivial, mas perde a exploração conforme n cresce;

- o **GA**, com a codificação por permutação e o OX, não aproveitou a recombinação.

O fator mais importante para o desempenho foi o casamento entre a vizinhança e a estrutura do problema (a *insertion*, que move um único ativo no tempo), mais do que a escolha da metaheurística.

Referências

[1] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer-Verlag, 2003.

[2] A. Gaspar-Cunha, R. H. C. Takahashi, and C. H. Antunes, editors. *Manual de Computação Evolutiva e Metaheurística*. Editora UFMG, 2013.

[3] David L. Poole and Alan K. Mackworth. *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press, 3 edition, 2023.

[4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 3 edition, 2016.

A Calibração do SA e do GA

Mediana do melhor h em 5 execuções por configuração (sementes 1000 a 1004 para o SA e 2000 a 2004 para o GA, distintas das do experimento principal). Em negrito, a melhor configuração por instância.

Tabela 5: Calibração do SA (χ_0: aceitação inicial da piora média; T_f: temperatura final).

Configuração	Pequena ($n = 12$)	Intermediária ($n = 50$)	Grande ($n = 150$)
χ_0=0.2 T_f=0.2	**1 875**	17 640	**67 586**
χ_0=0.2 T_f=1.0	**1 875**	16 755	69 362
χ_0=0.2 T_f=5.0	**1 875**	17 071	70 751
χ_0=0.5 T_f=0.2	**1 875**	16 854	69 694
χ_0=0.5 T_f=1.0	**1 875**	17 344	70 890
χ_0=0.5 T_f=5.0	**1 875**	16 665	71 289
χ_0=0.8 T_f=0.2	**1 875**	17 125	73 059
χ_0=0.8 T_f=1.0	**1 875**	**16 395**	69 715
χ_0=0.8 T_f=5.0	**1 875**	16 568	71 503

Tabela 6: Calibração do GA ($P_c = 0{,}9$, torneio 3, elitismo 2, OX).

Configuração	Pequena ($n = 12$)	Intermediária ($n = 50$)	Grande ($n = 150$)
N=100 P_m=0.2 mut.=insertion	2 230	18 249	99 638
N=100 P_m=0.2 mut.=swap	2 182	28 607	157 547
N=100 P_m=0.5 mut.=insertion	2 230	22 089	183 093
N=100 P_m=0.5 mut.=swap	2 182	27 440	205 872
N=100 P_m=1.0 mut.=insertion	**1 937**	24 774	237 883
N=100 P_m=1.0 mut.=swap	2 131	33 931	304 299
N=200 P_m=0.2 mut.=insertion	**1 937**	20 463	211 549
N=200 P_m=0.2 mut.=swap	2 256	29 441	179 015
N=200 P_m=0.5 mut.=insertion	**1 937**	32 303	341 330
N=200 P_m=0.5 mut.=swap	2 182	36 080	376 740
N=200 P_m=1.0 mut.=insertion	**1 937**	34 685	331 679
N=200 P_m=1.0 mut.=swap	**1 937**	43 062	389 136
N=50 P_m=0.2 mut.=insertion	2 125	19 960	91 397
N=50 P_m=0.2 mut.=swap	2 296	28 949	181 404
N=50 P_m=0.5 mut.=insertion	**1 937**	**17 704**	**88 542**
N=50 P_m=0.5 mut.=swap	2 230	26 022	150 973
N=50 P_m=1.0 mut.=insertion	**1 937**	20 557	150 283
N=50 P_m=1.0 mut.=swap	**1 937**	31 743	219 988

B Reprodução

O `README.md` do repositório lista os comandos que geram cada tabela e figura deste relatório:

```
python3 -m unittest discover -s tests   # testes (fitness, incremental, OX/PMX)
python3 src/experimentos.py todos       # grava resultados/*.csv e *_hist.json
python3 src/analise.py                  # gera relatorio/tabelas e figures/tp2
```